Humans and Agents in Software Engineering Loops

This article is part of "Exploring Gen AI". A series capturing Thoughtworks technologists' explorations of using gen ai technology for software development.

04 March 2026

Kief Morris

Kief Morris lives in London and works as a global cloud technology specialist for Thoughtworks.

Should humans stay out of the software development process and vibe code, or do we need developers in the loop inspecting every line of code? I believe the answer is to focus on the goal of turning ideas into outcomes. The right place for us humans is to build and manage the working loop rather than either leaving the agents to it or micromanaging what they produce. Let's call this "on the loop."

As software creators we build an outcome by turning our ideas into working software and iterating as we learn and evolve our ideas. This is the "why loop". Until the AI uprising comes humans will run this loop because we're the ones who want what it produces.

The process of building the software is the "how loop." The how loop involves creating, selecting, and using intermediate artefacts like code, tests, tools, and infrastructure. It may also involve documentation like technical designs and ADRs. We're used to seeing many of these as deliverables, but intermediate artefacts are really just a means to an end.

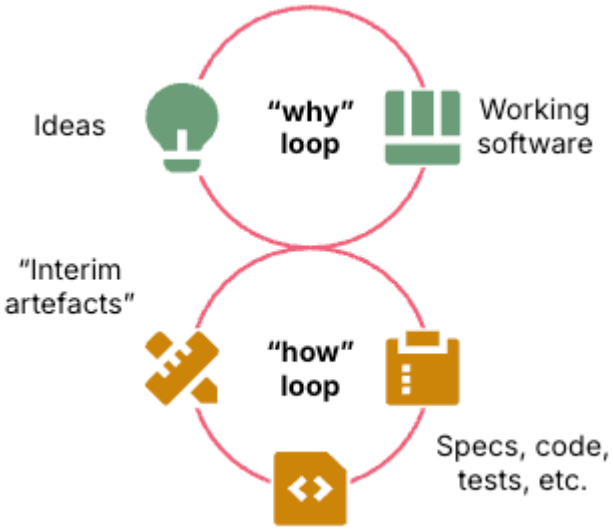

Figure 1: The why loop iterates over ideas and software, the how loop iterates on building the software

In reality the how loop contains multiple loops. The outermost how loop specifies and delivers the working software for the why loop. The innermost loop generates and tests code. Loops in between break down higher levels of work into smaller tasks for the lower loops to implement, then validate the results.

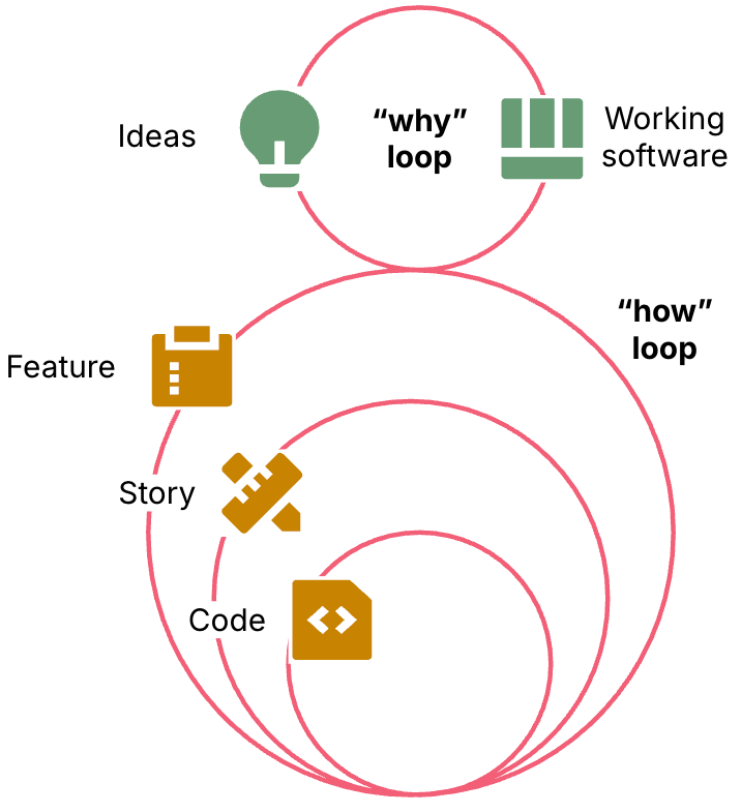

Figure 2: The how loop has multiple levels of inner loops that work on smaller increments of the full implementation

These loops may follow practices like design reviews and test stages. They might build systems by applying architectural approaches and design patterns like microservices or CUPID. Like the intermediate artefacts that pop out of these practices and patterns, they are all a means of achieving the outcome we actually care about.

But maybe we don't care about the means that are used to achieve our goals? Maybe we can just let the LLMs run the how loop however they like?

Humans outside the loop

Plenty of people have discovered the joy of letting humans stick to the why loop, and leaving the how loop for the agents to deal with. This is the common definition of "vibe coding". Some interpretations of Spec Driven Development (SDD) are much the same, with humans investing effort in writing the outcome we want, but not dictating how the LLM should achieve it.

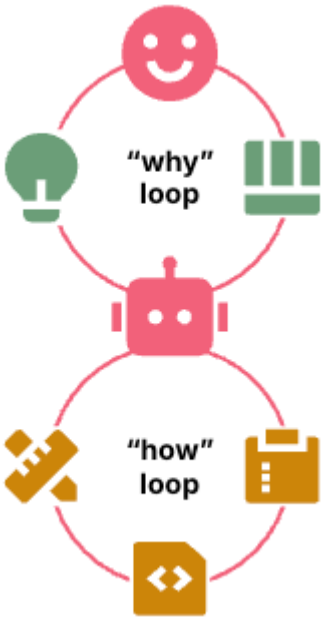

Figure 3: Human runs the why loop, agent runs the how loop.

The appeal of humans staying out of the how loop is that the why loop is the one we really care about. Software development is a messy domain that inevitably bogs down into over-engineered processes and coping with technical debt. And every new LLM model so far has gotten better at taking a user prompt and spitting out working software. If you're not satisfied with what it spits out, tell the LLM and it'll give you another iteration.

If the LLMs can write and change code without us, do we care whether the code is "clean"? It doesn't matter whether a variable name clearly expresses its purpose as long as an LLM can figure it out. Maybe we don't even need to care what language the software is written in?

We care about external quality, not internal quality for its own sake. External quality is what we experience as a user or other stakeholder of the software. Functional quality is a must, the system needs to work correctly. And for production software we also care about non-functional, operational quality. Our system shouldn't crash, it should run quickly, and we don't want it posting confidential data to social media sites. We don't want to run up massive cloud hosting bills, and in many domains we need to pass compliance audits.

We care about internal quality when it affects external outcomes. When human coders were crawling through the codebase, adding features and fixing bugs, they could do it more quickly and reliably in a clean codebase. But LLMs don't care about developer experience, do they?

In theory our LLM agents can extrude a massively overcomplicated spaghetti codebase, test and fix it by running ad-hoc shell commands, and eventually produce a correct, compliant, high-performing system. We just get our swarms Ralph Wiggumming on it, running in data centers that draw energy from the boiling oceans they float on, and eventually we'll get there. [1]

In practice, a cleanly-designed, well-structured codebase has externally important benefits over a messy codebase. When LLMs can more quickly understand and modify the code they work faster and spiral less. We do care about the time and cost of building the systems we need.

Humans in the loop

Some developers believe that the only way to maintain internal quality is to stay closely involved in the lowest levels of the how loop. Often, when an agent spirals over some broken bit of code a human developer can understand and fix it in seconds. Human experience and judgement still exceeds LLMs in many situations.

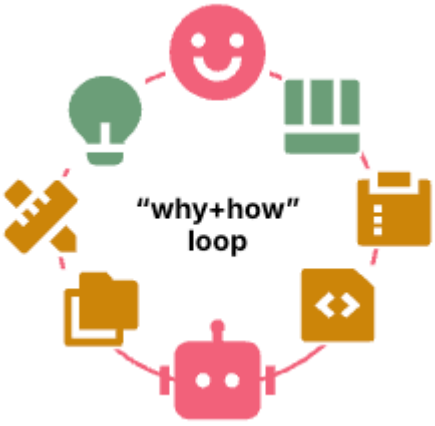

Figure 4: Human runs the why loop and the how loop

When people talk about “humans in the loop”, they often mean humans as a gatekeeper within the innermost loop where code is generated, such as manually inspecting each line of code created by an LLM.

The challenge when we insist on being too closely involved in the process is that we become a bottleneck. Agents can generate code faster than humans can manually inspect it. Reports on developer productivity with AI show mixed results, which may be at least partly because of humans spending more time specifying and reviewing code than they save by getting LLMs to generate it.

We need to adopt classic “shift left” thinking. Once upon a time we wrote all of our code, passed it to a QA team to test, and then tried to fix enough bugs to ship a release. Then we discovered that when developers write and run tests as we work we find and fix issues right away, which makes the whole process faster and more reliable.

What works for humans can work for agents as well. Agents produce better code when they can gauge the quality of the code they produce themselves rather than relying on us to check it for them. We need to instruct them on what we’re looking for, and give them guidance on the best ways to achieve it.

Humans on the loop

Rather than personally inspecting what the agents produce, we can make them better at producing it. The collection of specifications, quality checks, and workflow guidance that control different levels of loops inside the how loop is the agent’s harness. The emerging practice of building and maintaining these harnesses, Harness Engineering, is how humans work on the loop.

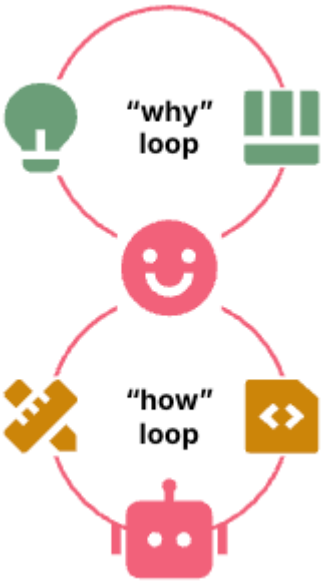

Figure 5: Human defines the how loop and the agent runs it

Something like the on the loop concept has also been described as the “middle loop,” including by participants of The Future of Software Development Retreat. The middle loop refers to moving human attention to a higher-level loop than the coding loop.

The difference between in the loop and on the loop is most visible in what we do when we’re not satisfied with what the agent produces, including an intermediate artefact. The “in the loop” way is to fix the artefact, whether by directly editing it, or by telling the agent to make the correction we want. The “on the loop” way is to change the harness that produced the artefact so it produces the results we want.

We continuously improve the quality of the outcomes we get by continuously improving the harness. And then we can take it to another level.

The agentic flywheel

The next level is humans directing agents to manage and improve the harness rather than doing it by hand.

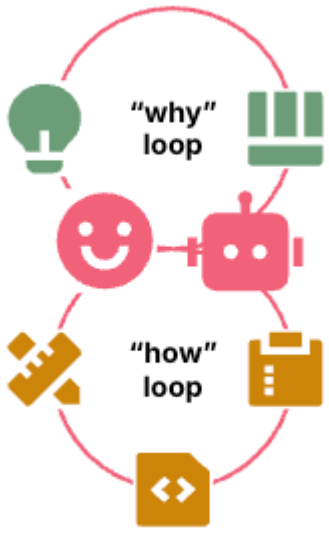

Figure 6: Human directs agent to build and improve the how loop

We build the flywheel by giving the agents the information they need to evaluate the performance of the loop. A good starting point is the tests and evaluations already included in the harness. The flywheel becomes more powerful as we feed it richer signals. Add pipeline stages that measure performance and validate failure scenarios. Feed operational data from production, user journey logs, and commercial results to broaden the scope and depth of what the agents can analyze.

For each step of the workflow we have the agent review the results and recommend improvements to the harness. The scope includes improvements to any of the

upstream parts of the workflow that could improve those results. What we have now is an agent harness that generates recommendations for improving itself.

We start by considering the recommendations interactively, prompting the agents to implement specific changes. We can also have the agents add their recommendations to the product backlog, so we can prioritize and schedule them for the agents to pick up, apply, and test as part of the automated flow.

As we gain confidence, the agents can assign scores to their recommendations, including the risks, costs, and benefits. We might then decide that recommendations with certain scores should be automatically approved and applied.

At some point this might look a lot like humans out of the loop, old-school vibe coding. I suspect that will be true for standard types of work that are done often as the improvement loops reach diminishing returns. But by engineering the harness we won't just get one-off, "good enough" solutions, we'll get robust, maybe even anti-fragile systems that continuously improve themselves.

1. These days "ralph loop" is often used colloquially to mean just firing up a bunch of agents and leaving them to keep looping until (hopefully) they finish their task. But as originally described the operator plays an important role in steering agents as they ralph. ↩

latest article (Mar 04):
Humans and Agents in Software Engineering Loops

previous article:
Harness Engineering - first thoughts

Architecture
Refactoring
Agile
Delivery
Microservices
Data
Testing
DSL
About
Books
FAQ
Videos
Content
Index
Fragments
Board
Games
Photography
RSS
Mastodon
LinkedIn
Bluesky
X
BGG
Home
Insights
Careers
Radar
Engineering